- Симуляция данных в numpy
- np.random: rand, randn, randint, choice, seed
- Broadcasting – что это и зачем?
- np.unique, np.sort, np.argsort
- Сериализация и сохранение данных

Программирование в лингвистике

Лекция 15

NumPy: продвинутый уровень

Симуляция данных в numpy

- Иногда нам нужно сгенерировать массив numpy с симулированными данными, чтобы проверить, как работает какая-то написанная нами функция.
- Модуль np.random используется для генерации случайных чисел.
- Базовые генераторы:

- rand(d0, d1, … dn) - генерирует массив заданной формы, заполненный числами из равномерного распределения на полуинтервале [0.0, 1.0).
- randn(d0, d1, ..., dn) - возвращает массив на основе стандартного нормального (гауссова) распределения с положительными и отрицательными числами.

```
import numpy as np

arr1 = np.random.rand(2, 2)
print("Равномерное:\n", arr1)

arr2 = np.random.randn(2, 2)
print("Нормальное:\n", arr2)
```

```
Равномерное:
[[0.35902345 0.74630193]
 [0.87124189 0.96748203]]
Нормальное:
[[ 0.07269545 -0.69735141]
 [-1.4660209   0.32927139]]
```

Симуляция данных в numpy

- randint(low, high=None, size=(d0, d1, … dn)) - генерирует массив случайных целых чисел с равномерным распределением от low (включительно) до high (не включая). Если high не указан, генерация идет в диапазоне [0, low).
- choice(a, size=None, replace=True, p=None) – выдает случайную выборку из одномерного массива или списка a.

  Параметр replace=True означает выбор с повторением; p позволяет вручную задать вероятность для каждого элемента.

Вероятности должны в сумме всегда давать 1

```
import numpy as np

fruits = ['apple', 'banana', 'orange']
selection = np.random.choice(fruits, size=5, p=[0.6, 0.3, 0.1])
print(selection)
```

```
['apple' 'banana' 'apple' 'apple' 'apple']
```

Повторяемость и seed

- `np.random.seed(value)` – после нее компьютер генерирует псевдослучайные числа на основе данного стартового числа (зерна).

Если зафиксировать seed, то при каждом перезапуске кода вы будете получать абсолютно одинаковые числа.

Это критически важно для повторяемости научных экспериментов и тестирования.

Внимание! Одинаковые числа получаются именно при перезапуске кода, а не при каждом вызове функции np.random

```
import numpy as np

np.random.seed(42)
arr1 = np.random.rand(2, 2)
print("Первая генерация:\n", arr1)

arr2 = np.random.rand(2, 2)
print("Вторая генерация n", arr2)
```

```
Первая генерация:
[[0.98690462 0.44030546]
 [0.74419187 0.99351124]]
Вторая генерация:
[[0.37454012 0.95071431]
 [0.73199394 0.59865848]]
```

Broadcasting

- **Broadcasting** (транслирование) — это автоматическое расширение размеров массивов при выполнении арифметических операций, позволяющее складывать или умножать массивы разных форм без явного копирования данных в памяти.

Правило совместимости размеров:

Два массива совместимы для транслирования, если, начиная с самых правых размеров (с конца), их размерности:

1. Полностью совпадают.
2. Либо одна из размерностей равна 1.

Если один из массивов имеет меньше размерностей, чем другой, его форма «достраивается» слева.

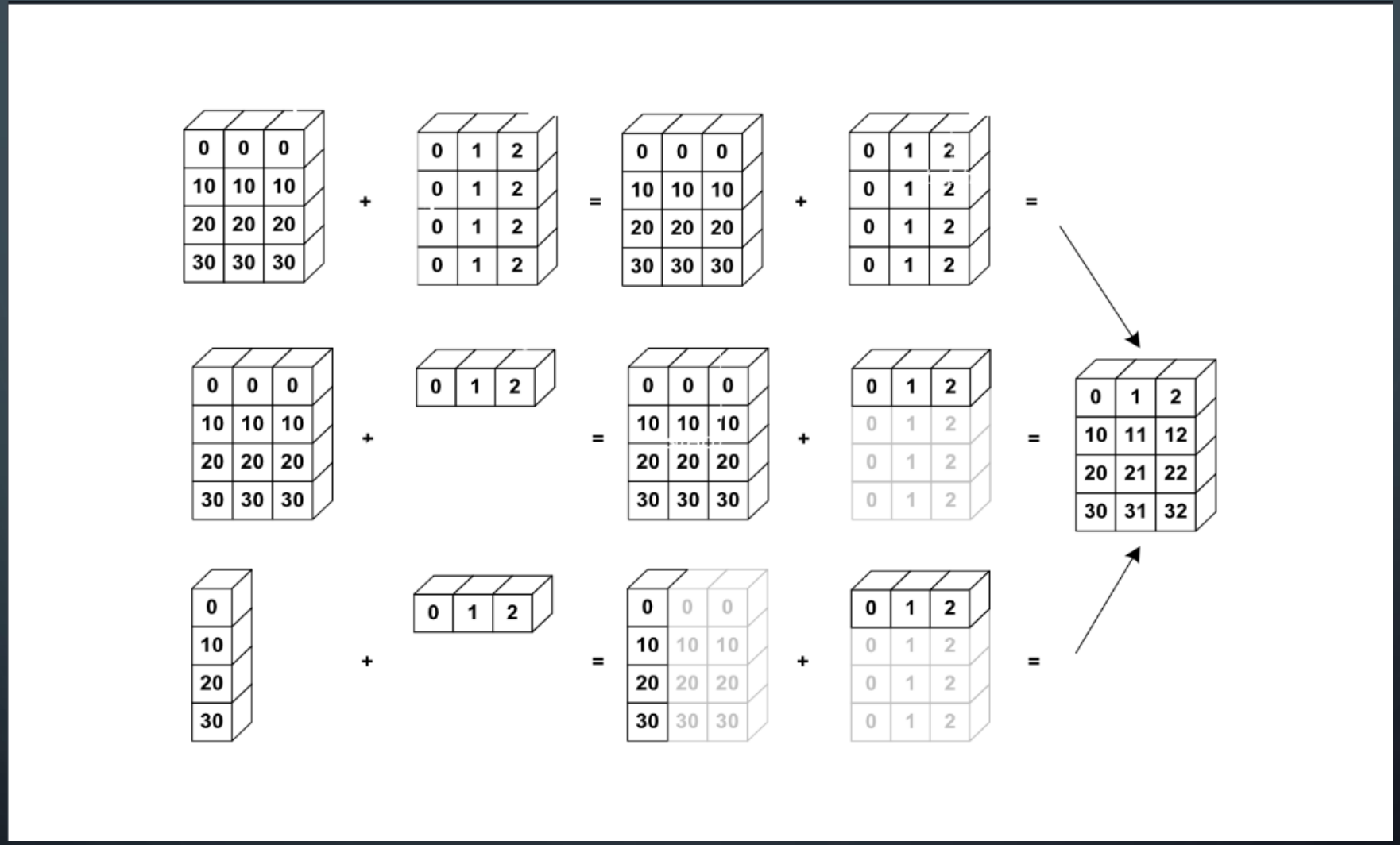


Иллюстрация: как происходит broadcasting

Утилиты для работы с данными

- `np.unique(arr, return_counts=False)` возвращает отсортированные уникальные элементы массива. Если передать `return_counts=True`, функция также вернет количество упоминаний каждого элемента.
- `np.sort(arr)` возвращает отсортированную копию массива по возрастанию.
- `np.argsort(arr)` возвращает индексы, которые расставили бы элементы массива в отсортированном порядке.
- `np.clip(arr, a_min, a_max)` - утилита для обработки выбросов данных и обрезки аномалий.

На практике в Data Science значения для границ в clip часто рассчитываются динамически на основе мат. статистики. Подробности см. в лекционной тетрадке.

```
import numpy as np

weights = np.array([80, 55, 95, 70])
indices = np.argsort(weights)
print(indices)
```

```
[1 3 0 2]
```

Сериализация и сохранение данных

Когда данные нужно сохранить на диск, встает выбор между pickle и .npy - родным для NumPy форматом.

Использование NumPy-формата:

- np.save('filename.npy', arr) сохраняет массив в бинарном формате .npy.
- np.load('filename.npy') – быстро загружает массив обратно в память.

Сравнение:

- Скорость: np.save и np.load работают значительно быстрее для больших числовых массивов. NumPy сбрасывает сырой буфер памяти напрямую на диск, минуя сложную сериализацию объектов.
- Размер: формат .npy хранит только заголовок с метаданными (форма, тип данных) и сам непрерывный блок байт. pickle добавляет избыточную служебную информацию.

Сериализация и сохранение данных

- Безопасность: pickle.load уязвим для выполнения произвольного вредоносного кода при загрузке ненадежных файлов. Формат .npy более изолирован и безопасен.

- Бонус: NumPy позволяет сохранение нескольких массивов в один файл.

np.savez('filename.npz', arr1, arr1, … arrn) упаковывает несколько массивов в один архив с расширением .npz. Массивы внутри архива сохраняются в виде словаря («ключ – значение»).

При загрузке такого файла мы получаем ленивый объект-контейнер.

np.savez_compressed('filename.npz', arr1, arr1, … arrn) работает так же, но дополнительно применяет алгоритм сжатия к файлам внутри архива.

СПАСИБО ЗА ВНИМАНИЕ!

Контакты для связи: ajumo@yandex.ru, abyssian03@gmail.com
GitHub: https://github.com/abyssian03/Stud-2